SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

EXPERIMENTS

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.

2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.

5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

RUBRICS FOR CALCULATION:

Q. No	Understanding of Concepts (25)	Experimental Design (30)	Use of Tools (20)	Analysis of Results (15)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 30**	**/ 20**	**/ 15**	**/ 10**	**/ 100**

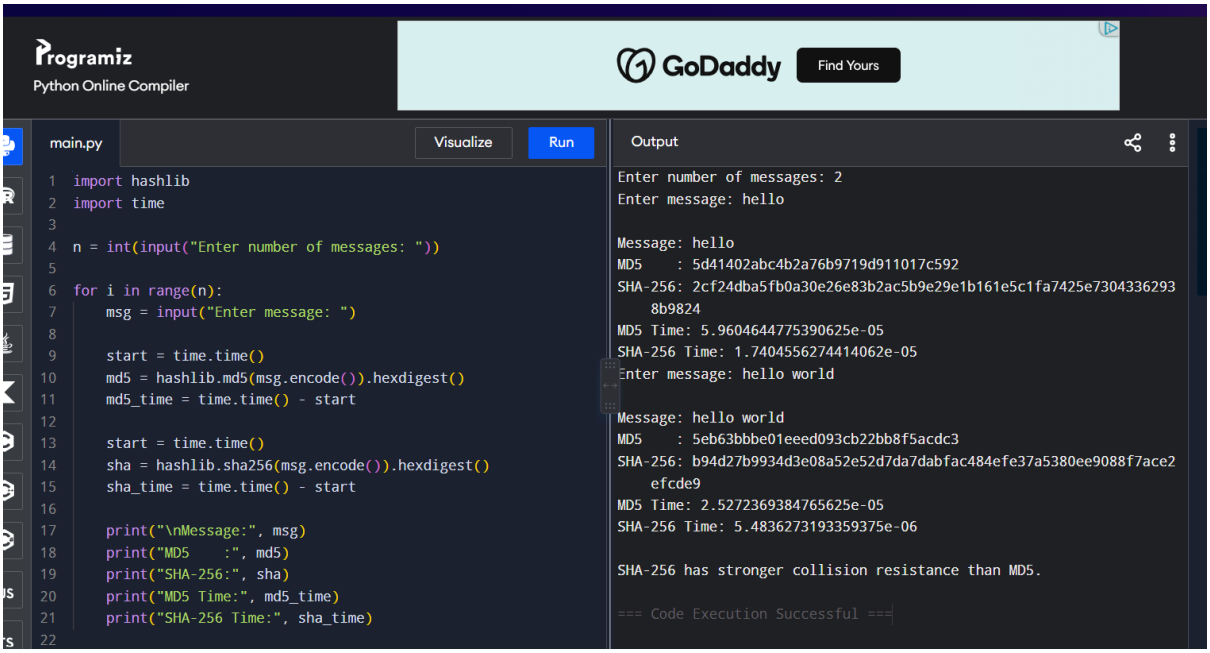
R. Deepasri (192511309)

CO3 AT1-Cryptography and NS(Experiments):

1. MD5 and SHA-256 Hashing

CODE:

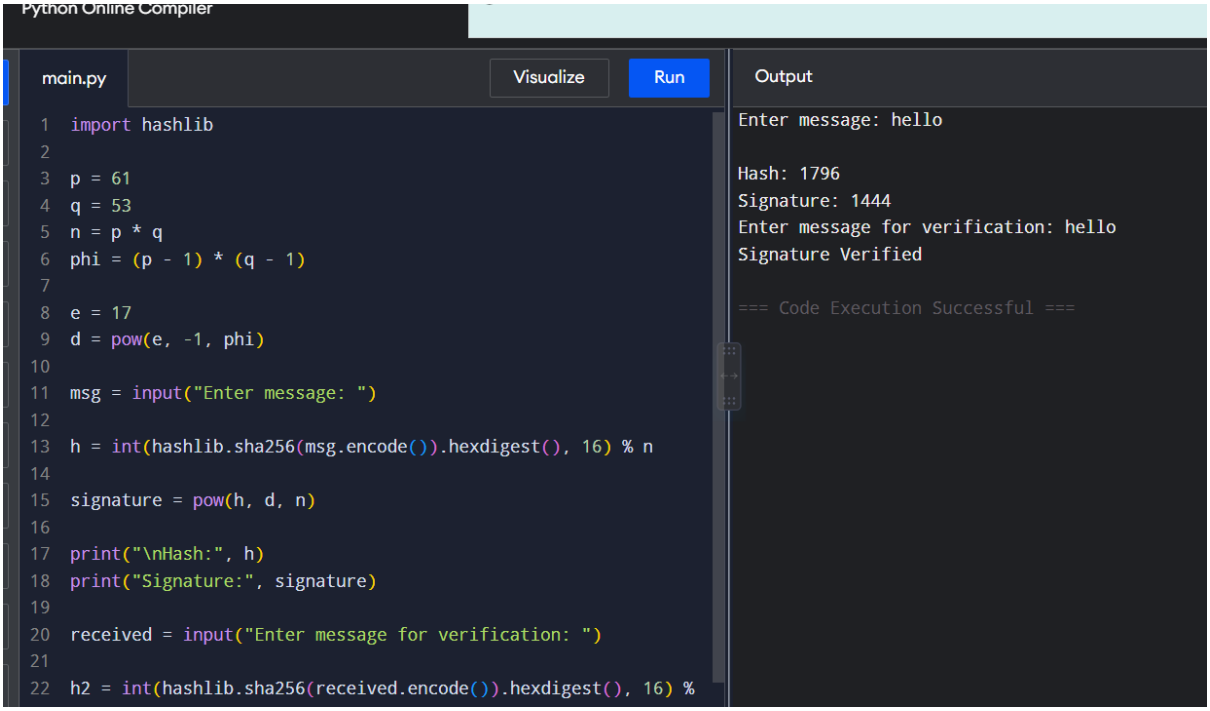
```
import hashlib
import time

n = int(input("Enter number of messages: "))

for i in range(n):
  msg = input("Enter message: ")

  start = time.time()
  md5 = hashlib.md5(msg.encode()).hexdigest()
  md5_time = time.time() - start

  start = time.time()
  sha = hashlib.sha256(msg.encode()).hexdigest()
  sha_time = time.time() - start

  print("\nMessage:", msg)
  print("MD5    :", md5)
  print("SHA-256:", sha)
  print("MD5 Time:", md5_time)
  print("SHA-256 Time:", sha_time)

print("\nSHA-256 has stronger collision resistance than MD5.")
```



2. RSA Digital Signature

```
import hashlib

p = 61
q = 53
n = p * q
phi = (p - 1) * (q - 1)

e = 17
d = pow(e, -1, phi)

msg = input("Enter message: ")

h = int(hashlib.sha256(msg.encode()).hexdigest(), 16) % n

signature = pow(h, d, n)

print("\nHash:", h)
print("Signature:", signature)

received = input("Enter message for verification: ")
```

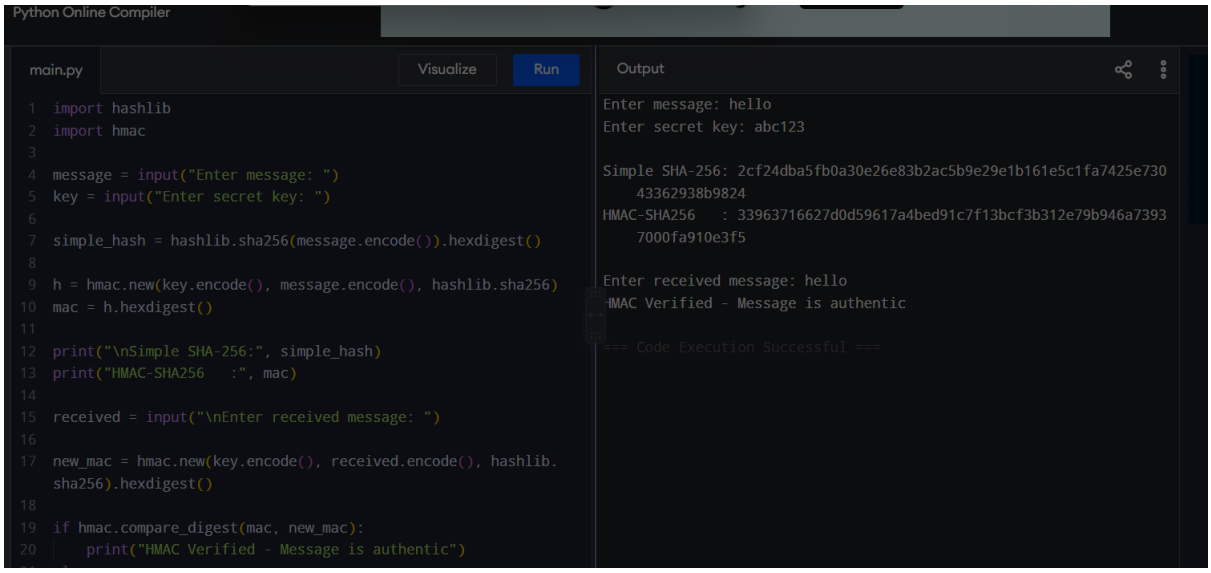
```
h2 = int(hashlib.sha256(received.encode()).hexdigest(), 16) % n
verify = pow(signature, e, n)

if h2 == verify:
   print("Signature Verified")
else:
   print("Signature Verification Failed")
```



3. HMAC

```
import hashlib
import hmac

message = input("Enter message: ")
key = input("Enter secret key: ")

simple_hash = hashlib.sha256(message.encode()).hexdigest()

h = hmac.new(key.encode(), message.encode(), hashlib.sha256)
mac = h.hexdigest()

print("\nSimple SHA-256:", simple_hash)
print("HMAC-SHA256   :", mac)
```

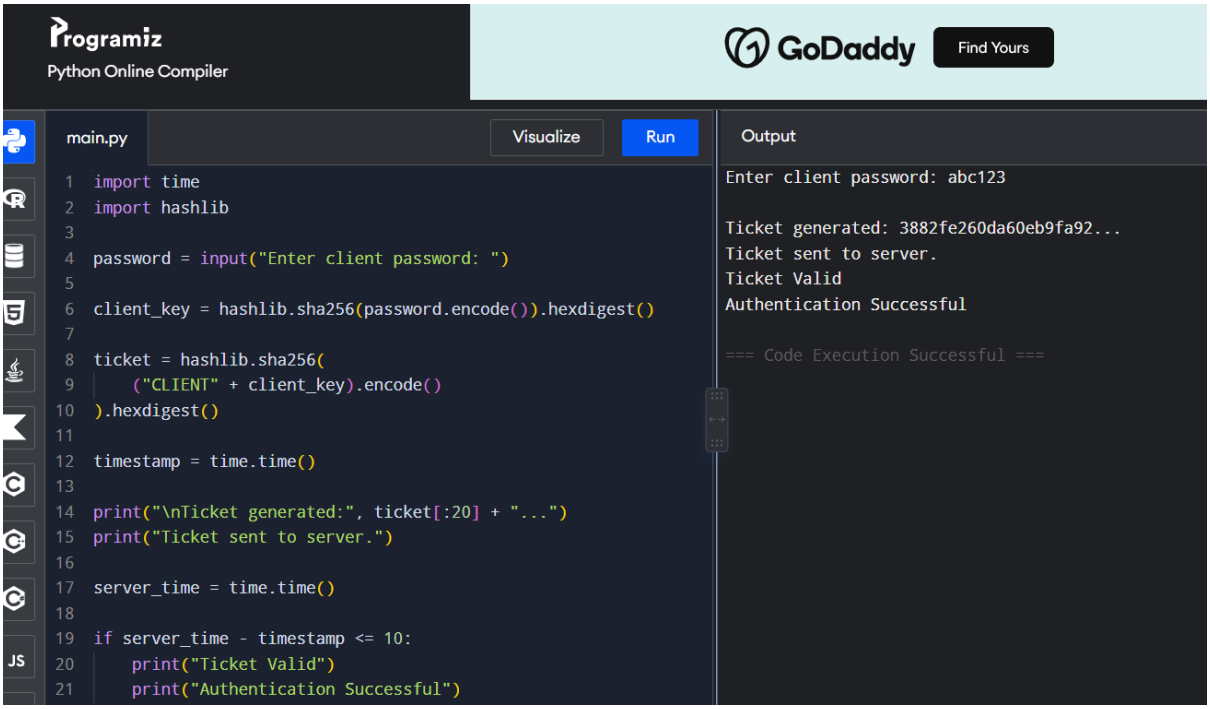
```
received = input("\nEnter received message: ")

new_mac = hmac.new(key.encode(), received.encode(),
hashlib.sha256).hexdigest()

if hmac.compare_digest(mac, new_mac):
   print("HMAC Verified - Message is authentic")
else:
   print("HMAC Failed - Message was modified")
```



4.Simplified Kerberos Authentication

```
import time
import hashlib

password = input("Enter client password: ")

client_key = hashlib.sha256(password.encode()).hexdigest()

ticket = hashlib.sha256(
   ("CLIENT" + client_key).encode()
).hexdigest()
```

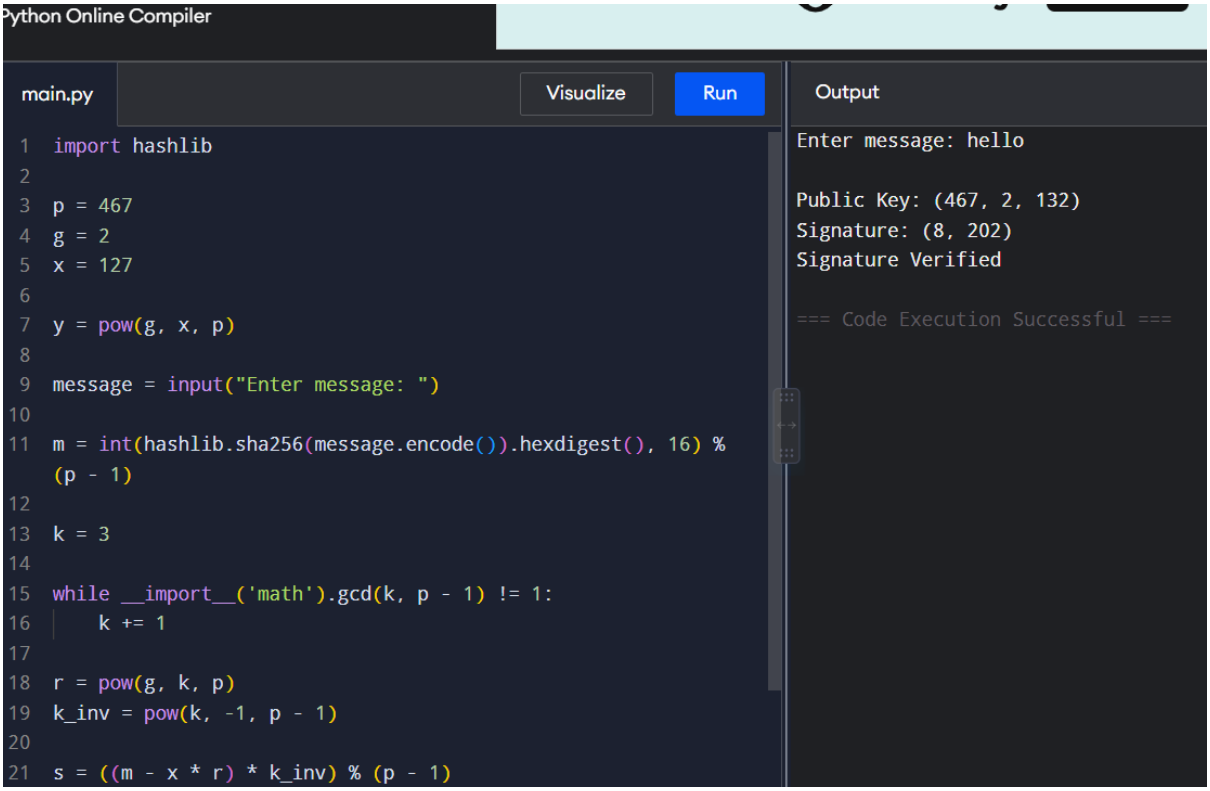
```
timestamp = time.time()

print("\nTicket generated:", ticket[:20] + "...")
print("Ticket sent to server.")

server_time = time.time()

if server_time - timestamp <= 10:
   print("Ticket Valid")
   print("Authentication Successful")
else:
   print("Ticket Expired")
   print("Authentication Failed")
```



5.ElGamal Digital Signature

```
import hashlib

p = 467
g = 2
x = 127

y = pow(g, x, p)
```

```
message = input("Enter message: ")

m = int(hashlib.sha256(message.encode()).hexdigest(), 16) % (p - 1)

k = 3

while __import__('math').gcd(k, p - 1) != 1:
  k += 1

r = pow(g, k, p)
k_inv = pow(k, -1, p - 1)

s = ((m - x * r) * k_inv) % (p - 1)

print("\nPublic Key:", (p, g, y))
print("Signature:", (r, s))

v1 = pow(y, r, p) * pow(r, s, p) % p
v2 = pow(g, m, p)

if v1 == v2:
  print("Signature Verified")
else:
  print("Signature Verification Failed")
```


Python Online Compiler
main.py
Visualize
Run
Output
import hashlib
p = 467
g = 2
x = 127
y = pow(g, x, p)
message = input("Enter message: ")
m = int(hashlib.sha256(message.encode()).hexdigest(), 16) % (p - 1)
k = 3
while __import__('math').gcd(k, p - 1) != 1:
k += 1
r = pow(g, k, p)
k_inv = pow(k, -1, p - 1)
s = ((m - x * r) * k_inv) % (p - 1)
Enter message: hello
Public Key: (467, 2, 132)
Signature: (8, 202)
Signature Verified
=== Code Execution Successful ===